WeatherForge Documentation

Real-Time vs. Dataset-Anchored Live Feed

The Concept
The "Live Feed" toggle on the WeatherForge dashboard can operate in one of two modes:

1. **Dataset-Anchored (Demo Mode):** The lookback window (1, 3, 6, or 12 months) is calculated backward from the *most recent storm event in the database*.
2. **Real-Time Anchored (Production Mode):** The lookback window is calculated backward from *today's actual calendar date* (`pd.Timestamp.now()`).

> **Why Real-Time is Not Currently Implemented:**
> The dashboard is currently running in **Dataset-Anchored (Demo Mode)**. Because the historical NOAA dataset (`storm_events_mn.parquet`) currently ends in late 2025, using a strict real-time anchor (e.g., looking back 1 month from May 2026) results in an empty dashboard. For portfolio demonstrations and presentations, it is critical that all UI controls return populated, interactive charts to showcase the application's analytical capabilities. The real-time logic has been temporarily suspended to ensure a seamless demo experience.

How to Update to Production (Real-Time) Mode

When the data pipeline is automated to pull fresh NOAA data regularly, the dashboard should be switched to Real-Time Mode. To do this, update the following two functions in `app.py` to replace the dataset maximums with `pd.Timestamp.now()` .

1. Update Data Filtering (`data_to_use`)

Locate the `data_to_use()` function and update the `if input.live_mode():` block. Replace `max_date` with `pd.Timestamp.now()` .

```
@reactive.Calc
def data_to_use():
    df = storms.copy()

    if STORM_DATE_COLUMN is None:
        return df.iloc[0:0].copy()

    if input.live_mode():
        period = input.live_period()
```

```
        days = {"1 Month": 30, "3 Months": 90, "6 Months": 180, "1 Year": 365}.get(period, 3

        # --- PRODUCTION CHANGE: Anchor to today's date ---
        current_date = pd.Timestamp.now()
        cutoff = current_date - pd.Timedelta(days=days)

        df = df[df[STORM_DATE_COLUMN] >= cutoff].copy()
    else:
        start_year, end_year = input.year_range()
        df = df[
            (df[STORM_DATE_COLUMN].dt.year >= start_year)
            & (df[STORM_DATE_COLUMN].dt.year <= end_year)
        ].copy()

    # ... (keep existing county_filter logic)
    return df
```

2. Update Population Math (`selected_year_range`)

Locate the `selected_year_range()` function. This must be updated simultaneously so the per-capita risk score normalizes against the correct census year.

```
@reactive.Calc
def selected_year_range() -> tuple[int, int]:
    '''Returns (start_year, end_year) for the current sidebar selection.'''
    if input.live_mode():
        days = {"1 Month": 30, "3 Months": 90, "6 Months": 180, "1 Year": 365}[input.live_pe

        # --- PRODUCTION CHANGE: Anchor to today's date ---
        current_date = pd.Timestamp.now()
        end = current_date.year
        start = (current_date - pd.Timedelta(days=days)).year
        return start, end

    start, end = input.year_range()
    return int(start), int(end)
```